Internal Pipes Were All You Needed Too

A Wallet-Gated, Layer-Ascending Substrate for Agent Computation: Composition, Approval, and Capability across the Stack

The inverse of "Internal APIs Were Not All You Needed." Where that paper DESCENDS one signed stack — screen → browser → CLI → OS → kernel → packet — so a single agent reaches outward and downward to ACT on a hostile external surface, this paper ASCENDS a composition — binary → pipe → binary → … → capability — so many trusted local primitives compose upward into one capability. Forward: one key SIGNS every layer (admission to act, push). Inverse: one wallet APPROVES every value crossing every pipe (toll to release, pull). The two papers are one shape walked in opposite directions; together they are the full stack the first paper said it had only half-shipped.

Abstract

The companion paper made one claim and ran it down a signed stack: the discipline that makes a single request trustworthy — sign it, seal it, cache it, prove who you are without showing your hand — holds at every layer of a *descent* to act on the web. It also conceded, in plain print, that "real signed OS/kernel/packet descent across platforms" remained integration work. This paper closes that frontier from the other end. Instead of descending one agent's own layers to emit a packet, we *compose* the operating system's own binaries upward, where each binary is a contract neuron, each unix pipe `A | B` is a content-addressed pointer edge, and each value crossing a pipe is released only under a real Ed25519 wallet approval — sealed (pointer, not payload) until the capability opens it. The same covenant — content-address, witness, seal, ledger — holds, but the verb inverts: *descent signs to act outward; ascent approves to release inward.* We cite a real primitive for each layer rather than inventing one, and we label what is **[shipped]** (a runnable witness whose four claims pass as executable tests), what is **[reference]** (the cited foundations), and what is **[proposed]** (the elizaOS wiring) so the reader always knows whether they are looking at running code or a promise. The short version: internal APIs were a great first layer, signed descent was the second — and *composed, wallet-gated pipes* are the floor both were standing on.

1. Introduction: the stack runs both ways

The companion paper's descent answers *how does one agent act, coherently, all the way down to the wire.* It is the right question when the surface is hostile and external — a login form, a bot-detector, a TLS fingerprint. But most of an agent's real work is not reaching out to a hostile web; it is composing trusted local capability: shelling a binary, piping its output to another, reading a file, calling a syscall. There the threat is not "does the wire believe I am a browser" but "should this value be allowed to cross from this binary into that one, and who approved it."

So we run the same end-to-end argument [e2e] the other way. The descent places a function at the *highest* layer that can do it completely and falls back downward; the ascent *composes* a capability from the *lowest* binaries upward and gates each composition. The unit inverts with the direction: the descent's unit is an OSI layer, the ascent's unit is a binary — a process that reads stdin, writes stdout, and does one thing well [unixprog]. And the trust primitive inverts with it: the descent *signs* to prove admission to act; the ascent *approves* to release a value across a boundary. A signature is a push (here is proof I may act); an approval is a pull (here is the toll to let this value through).

The thesis fits on one line: *the layering is still the point, but composition is its other direction, and*

the covenant holds going up as it held going down.

2. The stack as a composing tree

We model the local stack not as a descent line but as a DAG of binaries joined by pipes:

```
binary -> pipe -> binary -> pipe -> ... -> agent capability
```

Each binary is a contract neuron. Its identity is the content-address of its output; its parent is the upstream binary whose value it consumed; its witness is the approval that released that value; its verb is read/route/write. A pipe `A | B` is a `contract:<id>` cross-edge: B's stdin neuron points at A's stdout neuron. And the tree is self-similar — a named pipeline is itself a binary, so the shape repeats from a single `echo | wc` up to a whole agent capability, exactly as content-addressed build systems compose derivations that name their inputs by hash [nix]. The reason any of this composes is the same reason the descent did: the shape at every layer is identical — produce, content-address, approve, release — so *one* wallet and *one* content-addressed cache serve the entire tree instead of N bespoke integrations.

Each rung of the ascent is one atom of the covenant tree, mapped to an existing, cited primitive (Table 1). We did not invent these; we aligned them.

Table 1. Each layer of the composing stack is one atom of the covenant tree, mapped to an existing primitive — the inverse of the companion paper's Table 1.

atom	composing-stack primitive	source
root (why)	object-capability model — authority by reference, never ambient	Miller, *Robust Composition* [ocap]
node (what)	the unix process as a filter — one binary, one thing	Pike & Kernighan [unixprog]
tree (where)	the Nix build DAG — derivations content-addressed, naming inputs by hash	Dolstra [nix]
verb (how)	propagator moves — write-pipe / route / read-pipe	Radul [propagator]
settle (when)	bounded pipe + SIGPIPE — drain (corroboration) or broken pipe (clock)	POSIX pipe(7) [pipe7]
witness (who)	the macaroon — bearer approval, verifiable, attenuatable, leaks nothing	Birgisson et al. [macaroons]
cache (value)	content-addressed action cache — identical pipeline = cache hit	Bazel Remote Execution [bazelcas]
seal (no unsealed)	AEAD at the IPC boundary — sealed, fails closed	RFC 5116 [aead]
walk (where)	make's dependency walk — lowest-first, stale only	POSIX make [make]

atom	composing-stack primitive	source
loop (control)	the supervision tree (let-it-crash) — restart, converge	Armstrong [erlang]

3. Root: one wallet approves every release

The companion paper's root is an Ed25519 key whose *signature* is admission to act; the descent is signed top to bottom. The ascent's root is the same unforgeability turned the other way — the object-capability model [ocap]: authority originates at one unforgeable reference and travels *only by reference*, never as ambient permission. A value crosses a pipe only when the capability to release it is presented. The axiom never re-proven is "no capability, no release." The wallet is the same wallet — the agent already has an Ed25519 keypair — but its job inverts from *prove who acts* to *approve what is released.*

4. Witness without disclosure: the macaroon at the pipe

The descent's hard half was proving a credential belongs to the identity *without revealing the credential* — zkLogin and Pedersen commitments binding "that a credential is bound, never what it is." The ascent's hard half is the mirror: approving that a value *may cross a pipe* without exposing the value. The macaroon [macaroons] is the fit — a bearer token with caveats, independently verifiable and attenuatable, that rides along the pipe. The release is corroborated by two independent things: the producer's content-address (the cid proves *what*) and the approver's signed macaroon over that cid (the wallet proves *may-release*) — and the verifier confirms the binding without ever dereferencing the plaintext. Two witnesses, no disclosure, going up instead of down.

5. Seal and cache: sealed pipes, replayed compositions

Every value at rest between two binaries is content-addressed (the cid is the sha256 of its own bytes) and sealed with AEAD under the approver's key [aead]: the receiver checks the tag before trusting a single byte, failing closed on tamper rather than leaking into logs. This is the companion paper's `wallet-seal` and AEAD-at-the-wire, applied to the *pipe boundary* instead of the network boundary. And the cache is the same content-addressed store, inverted to memoize *compositions* rather than descents: an action keyed by the hash of its inputs returns its cached output [bazelcas], so re-running an identical pipeline is a cache hit — a miss rebuilds the value, a hit replays it, exactly like a Docker or Nix layer cache. The descent cached an action it had already signed; the ascent caches a value it has already approved. Same discipline, other direction.

6. The control loop

The whole composition runs under a supervisor [erlang]: a stage that exits nonzero or whose approval is denied is a failure the supervisor restarts or escalates, and the composition converges as the final stage exits 0. It is the same Plan → Build → Test → Judge cycle the descent ran, walked up the tree: Plan the pipeline, Build it stage by stage, Test the final exit, Judge the value, repent on failure. The glamorous control loops are the ones that do not converge; this one does, because every stage either produces an approved, content-addressed value or fails closed.

7. Evaluation: the composing substrate, by executable test

The load-bearing claim — *a unix pipe is a wallet-approval-gated contract pointer* — is not asserted; it is [**shipped**] as a runnable witness (`pipe_contract.py`) that composes the real OS binaries `echo | tr | wc` through the gate using real sha256 content-addressing, real Ed25519 approval signatures, an append-only hash-chained ledger, and a blob store. It exits 0 iff all four claims hold, and does so on two cold runs:

1. **Inline-free** — the pipe edge records carry only `{cid, len}`; the plaintext appears in zero ledger rows.
2. **Fail-closed** — drop a binary from the wallet's allowlist and the value is never dereferenced; a `denied` row is written and the pipeline halts.
3. **Reproduction** — an approved hop reproduces the upstream's exact bytes (the sha256 of the dereferenced blob equals the produced cid).
4. **Cache-hit** — re-running the identical pipeline yields identical cids and reuses blobs from the store.

What this establishes, and what it does not: the witness establishes that the composing substrate's primitive is correct and that approval gates releases as specified. It does not establish end-to-end agent task success, nor a hardened multi-fan-in DAG, nor real syscall-level neurons across platforms — those are out of this paper's scope, which is the composing substrate alone.

8. Threat model (the inverse of the descent's)

We name the adversary, because a trust paper that names nothing it defends against is decoration. The descent defended against a Dolev–Yao network attacker watching the wire; the ascent defends against value crossing a process boundary it should not.

- **(B1) The exfiltrator.** A value escapes a binary it should not have left. Resisted at *witness/seal*: no value crosses a pipe without a signed approval, and at rest it is AEAD-sealed and content-addressed, so an unapproved hop releases nothing. Residual: a binary may leak its own input through a side channel it controls — we gate the pipe, not the process's conscience.
- **(B2) The confused deputy.** A binary is tricked into releasing a value under another's authority. Resisted at *root*: authority is a capability, never ambient, so a downstream that lacks the macaroon cannot pull the value, and capability attenuation bounds what a delegated release may do.
- **(B3) The replayer.** An old approval is re-used to release a new value. Resisted at *ledger*: every release is a hash-chained row, so a reordered or duplicated approval is detectable. Residual: key theft is the holder, by construction — the same boundary every signing system accepts.

What we do not defend: a malicious binary that lies in its own output (a contract can faithfully replay a hostile filter), a global passive observer correlating process timing, or coercion of the key holder. Out of scope and named so the reader is not misled by silence.

9. What is built, what is referenced (no fabricated green)

- [**shipped**] The composing-substrate primitive: real OS binaries composed through a wallet-approval gate, content-addressed values, Ed25519 approvals, a hash-chained ledger, a blob store — with the four-claim witness passing as an executable test on two cold runs.
- [**reference**] Each cited foundation in Table 1 — object-capabilities, the unix filter, the Nix DAG, propagators, the bounded pipe, macaroons, the content-addressed action cache, AEAD,

make's walk, the supervision tree — is real prior art, not invented here.
- **[proposed]** The elizaOS wiring: an additive `plugin-contract-pipe` service that wraps the engine's existing subprocess and native-bridge calls so every `A | B` content-addresses A's output, gates the cid on a wallet approval for downstream B, and records the `piped/denied` edge — turning every OS binary an agent shells into a contract neuron, the object-capability OS layer the companion paper cited but did not ship. Integration work, honestly labelled.

10. Conclusion

Internal APIs were an excellent first layer; signed descent was the second; and both were standing on a third — *composition*. An agent that is sovereign over its own actions has to be one coherent entity not only reaching outward across the layers it descends, but composing inward across the binaries it joins, releasing each value only under a capability it can actually withhold. The contribution is deliberately modest and exactly inverse: one covenant — content-address, witness, seal, ledger — that held going down also holds going up, with the verb flipped from sign-to-act to approve-to-release, each layer pinned to a real primitive. Internal APIs were a great deal of what you needed [apis], cryptography was what you needed for security — and *composed, wallet-gated pipes* are how the two reach all the way down to the binaries, where each one is a pointer to the next.

References

- [apis] L. Tham et al. *Internal APIs Are All You Need.* arXiv:2604.00694, 2026.
- [e2e] J. H. Saltzer, D. P. Reed, D. D. Clark. *End-to-End Arguments in System Design.* ACM TOCS 2(4), 1984.
- [ocap] M. S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control.* PhD thesis, JHU, 2006. http://www.erights.org/talks/thesis/markm-thesis.pdf
- [unixprog] R. Pike, B. Kernighan. *Program Design in the UNIX Environment.* 1984. https://harmful.cat-v.org/cat-v/unix_prog_design.pdf
- [nix] E. Dolstra. *The Purely Functional Software Deployment Model.* PhD thesis, 2006. https://edolstra.github.io/pubs/phd-thesis.pdf
- [propagator] A. Radul. *Propagation Networks: A Flexible and Expressive Substrate for Computation.* MIT, 2009. https://dspace.mit.edu/handle/1721.1/44215
- [pipe7] *pipe(7) — Linux manual.* https://man7.org/linux/man-pages/man7/pipe.7.html
- [macaroons] A. Birgisson et al. *Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud.* NDSS 2014. github:rescrv/libmacaroons
- [bazelcas] *Bazel Remote Execution API.* github:bazelbuild/remote-apis
- [aead] D. McGrew. *An Interface and Algorithms for Authenticated Encryption.* RFC 5116, 2008. https://www.rfc-editor.org/rfc/rfc5116
- [make] *make — POSIX.1-2017.* https://pubs.opengroup.org/onlinepubs/9699919799/utilities/make.html
- [erlang] J. Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors.* PhD thesis, 2003. https://erlang.org/download/armstrong_thesis_2003.pdf